4 Working with nonrelational data

This chapter covers

- Persisting data to Cassandra
- Data modeling in Cassandra
- Working with document data in MongoDB

They say that variety is the spice of life.

You probably have a favorite flavor of ice cream. It's that one flavor that you choose the most often because it satisfies that creamy craving more than any other. But most people, despite having a favorite flavor, try different flavors from time to time to mix things up.

Databases are like ice cream. For decades, the relational database has been the favorite flavor for storing data. But these days, we have more options available than ever before. So-called "NoSQL" databases (**https://aws.amazon.com/nosql/**) offer different concepts and structures in which data can be stored. And although the choice may still be somewhat based on taste, some databases are better suited for persisting different kinds of data than others.

Fortunately, Spring Data has you covered for many of the NoSQL databases, including MongoDB, Cassandra, Couchbase, Neo4j, Redis, and many more. And fortunately, the programming model is nearly identical, regardless of which database you choose.

There's not enough space in this chapter to cover all of the databases that Spring Data supports. But to give you a sample of Spring Data's other "flavors," we'll look at two popular NoSQL databases, Cassandra and MongoDB, and see how to create repositories to persist data to them. Let's start by looking at how to create Cassandra repositories with Spring Data.

4.1 Working with Cassandra repositories

Cassandra is a distributed, high-performance, always available, eventually consistent, partitioned-column-store, NoSQL database.

That's a mouthful of adjectives to describe a database, but each one accurately speaks to the power of working with Cassandra. To put it in simpler terms, Cassandra deals in rows of data written to tables, which are partitioned across one-to-many distributed nodes. No single node carries all the data, but any given row may be replicated across multiple nodes, thus eliminating any single point of failure.

Spring Data Cassandra provides automatic repository support for the Cassandra database that's quite similar to—and yet quite different from—what's offered by Spring Data JPA for relational databases. In addition, Spring Data Cassandra offers annotations for mapping application domain types to the backing database structures.

Before we explore Cassandra any further, it's important to understand that although Cassandra shares many concepts similar to relational databases like Oracle and SQL Server, Cassandra isn't a relational database and is in many ways quite a different beast. I'll explain the idiosyncrasies of Cassandra as they pertain to working with Spring Data. But I encourage you to read Cassandra's own documentation (**http://cassandra.apache.org/doc/latest/**) for a thorough understanding of what makes it tick.

Let's get started by enabling Spring Data Cassandra in the Taco Cloud project.

4.1.1 Enabling Spring Data Cassandra

To get started using Spring Data Cassandra, you'll need to add the Spring Boot starter dependency for nonreactive Spring Data Cassandra. There are actually two separate Spring Data Cassandra starter dependencies to choose from: one for reactive data persistence and one for standard, nonreactive persistence.

We'll talk more about writing reactive repositories later in chapter 15. For now, though, we'll use the nonreactive starter in our build as shown here:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra</artifactId>
</dependency>
```

This dependency is also available from the Initializr by checking the Cassandra check box.

It's important to understand that this dependency is in lieu of the Spring Data JPA starter or Spring Data JDBC dependencies we used in the previous chapter. Instead of persisting Taco Cloud data to a relational database with JPA or JDBC, you'll be using Spring Data to persist data to a Cassandra database. Therefore, you'll want to remove the Spring Data JPA or Spring Data JDBC starter dependencies and any relational database dependencies (such as JDBC drivers or the H2 dependency) from the build.

The Spring Data Cassandra starter dependency brings a handful of dependencies to the project, specifically, the Spring Data Cassandra library. As a result of Spring Data Cassandra being in the runtime classpath, autoconfiguration for creating Cassandra repositories is triggered. This means you're able to begin writing Cassandra repositories with minimal explicit configuration.

Cassandra operates as a cluster of nodes that together act as a complete database system. If you don't already have a Cassandra cluster to work with, you can start a single-node cluster for development purposes using Docker like this:

```
$ docker network create cassandra-net
$ docker run --name my-cassandra \
        --network cassandra-net \
        -p 9042:9042 \
        -d cassandra:latest
```

This starts the single-node cluster and exposes the node's port (9042) on the host machine so that your application can access it.

You'll need to provide a small amount of configuration, though. At the very least, you'll need to configure the name of a keyspace within which your repositories will operate. To do that, you'll first need to create such a keyspace.

NOTE In Cassandra, a *keyspace* is a grouping of tables in a Cassandra node. It's roughly analogous to how tables, views, and constraints are grouped in a relational database.

Although it's possible to configure Spring Data Cassandra to create the keyspace automatically, it's typically much easier to manually create it yourself (or to use an existing keyspace). Using the Cassandra CQL (Cassandra Query Language) shell, you can create a keyspace for the Taco Cloud application. You can start the CQL shell using Docker like this:

```
$ docker run -it --network cassandra-net --rm cassandra cqlsh my-cassandra
```

NOTE If this command fails to start up the CQL shell with an error indicating "Unable to connect to any servers," wait a minute or two and try again. You need to be sure that the Cassandra cluster is fully started before the CQL shell can connect to it.

When the shell is ready, use the `create keyspace` command like this:

```
cqlsh> create keyspace tacocloud
   ... with replication={'class':'SimpleStrategy', 'replication_factor':1}
   ... and durable_writes=true;
```

Put simply, this will create a keyspace named `tacocloud` with simple replication and durable writes. By setting the replication factor to `1`, you ask Cassandra to keep one copy of each row. The replication strategy determines how replication is handled. The `SimpleStrategy` replication strategy is fine for single data center use (and for demo code), but you might consider the `NetworkTopologyStrategy` if you have your Cassandra cluster spread across multiple data centers. I refer you to the Cassandra documentation for more details of how replication strategies work and alternative ways of creating keyspaces.

Now that you've created a keyspace, you need to configure the `spring.data.cassandra.keyspace-name` property to tell Spring Data Cassandra to use that keyspace, as shown next:

```
spring:
  data:
    cassandra:
      keyspace-name: taco_cloud
      schema-action: recreate
      local-datacenter: datacenter1
```

Here, you also set the `spring.data.cassandra.schema-action` to `recreate`. This setting is very useful for development purposes because it ensures that any tables and user-defined types will be dropped and recreated every time the application starts. The default value, `none`, takes no action against the schema and is useful in production settings where you'd rather not drop all tables whenever an application starts up.

Finally, the `spring.data.cassandra.local-datacenter` property identifies the name of the local data center for purposes of setting Cassandra's load-balancing policy. In a single-node setup, `"datacenter1"` is the value to use. For more information on Cassandra load-balancing policies and how to set the local data center, see the DataStax Cassandra driver's reference documentation (**http://mng.bz/XrQM**).

These are the only properties you'll need for working with a locally running Cassandra database. In addition to these two properties, however, you may wish to set others, depending on how you've configured your Cassandra cluster.

By default, Spring Data Cassandra assumes that Cassandra is running locally and listening on port 9042. If that's not the case, as in a production setting, you may want to set the `spring.data.cassandra.contact-points` and `spring.data.cassandra.port` properties as follows:

```
spring:
  data:
    cassandra:
      keyspace-name: tacocloud
      local-datacenter: datacenter1
      contact-points:
      - casshost-1.tacocloud.com
      - casshost-2.tacocloud.com
      - casshost-3.tacocloud.com
      port: 9043
```

Notice that the `spring.data.cassandra.contact-points` property is where you identify the hostname(s) of Cassandra. A contact point is the host where a Cassandra node is running. By default, it's set to `localhost`, but you can set it to a list of hostnames. It will try each contact point until it's able to connect to one. This is to ensure that there's no single point of failure in the Cassandra cluster and that the application will be able to connect with the cluster through one of the given contact points.

You may also need to specify a username and password for your Cassandra cluster. This can be done by setting the `spring.data.cassandra.username` and `spring.data.cassandra.password` properties, as shown next:

```
spring:
  data:
    cassandra:
      ...
```

```
username: tacocloud
password: s3cr3tP455w0rd
```

Here the `spring.data.cassandra.username` and `spring.data.cassandra.password` properties specify "tacocloud" and "s3cr3tP455w0rd" as the credentials needed to access the Cassandra cluster.

Now that Spring Data Cassandra is enabled and configured in your project, you're almost ready to map your domain types to Cassandra tables and write repositories. But first, let's step back and consider a few basic points of Cassandra data modeling.

1.2 Understanding Cassandra data modeling

As I mentioned, Cassandra is quite different from a relational database. Before you can start mapping your domain types to Cassandra tables, it's important to understand a few of the ways that Cassandra data modeling is different from how you might model your data for persistence in a relational database.

A few of the most important things to understand about Cassandra data modeling follow:

- Cassandra tables may have any number of columns, but not all rows will necessarily use all of those columns.
- Cassandra databases are split across multiple partitions. Any row in a given table may be managed by one or more partitions, but it's unlikely that all partitions will have all rows.
- A Cassandra table has two kinds of keys: partition keys and clustering keys. Hash operations are performed on each row's partition key to determine which partition(s) that row will be managed by. Clustering keys determine the order in which the rows are maintained within a partition (not necessarily the order in which they may appear in the results of a query). Refer to Cassandra documentation (**http://mng.bz/yJ6E**) for a more detailed explanation of data modeling in Cassandra, including partitions, clusters, and their respective keys.
- Cassandra is highly optimized for read operations. As such, it's common and desirable for tables to be highly denormalized and for data to be duplicated across multiple tables. (For example, customer information may be kept in a customer table as well as duplicated in a table containing orders placed by customers.)

Suffice it to say that adapting the Taco Cloud domain types to work with Cassandra won't be a matter of simply swapping out a few JPA annotations for Cassandra annota-

tions. You'll have to rethink how you model the data.

1.3 Mapping domain types for Cassandra persistence

In chapter 3, you marked up your domain types (`Taco` , `Ingredient` , `TacoOrder` , and so on) with annotations provided by the JPA specification. These annotations mapped your domain types as entities to be persisted to a relational database. Although those annotations won't work for Cassandra persistence, Spring Data Cassandra provides its own set of mapping annotations for a similar purpose.

Let's start with the `Ingredient` class, because it's the simplest to map for Cassandra. The new Cassandra-ready `Ingredient` class looks like this:

```
package tacos;

import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Table("ingredients")
public class Ingredient {

  @PrimaryKey
  private String id;
  private String name;
  private Type type;

  public enum Type {
    WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
  }

}
```

The `Ingredient` class seems to contradict everything I said about just swapping out a few annotations. Rather than annotating the class with `@Entity` as you did for JPA per-

sistence, it's annotated with `@Table` to indicate that ingredients should be persisted to a table named `ingredients`. And rather than annotate the `id` property with `@Id`, this time it's annotated with `@PrimaryKey`. So far, it seems that you're only swapping out a few annotations.

But don't let the `Ingredient` mapping fool you. The `Ingredient` class is one of your simplest domain types. Things get more interesting when you map the `Taco` class for Cassandra persistence, as shown in the next listing.

Listing 4.1 Annotating the `Taco` class for Cassandra persistence

```
package tacos;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.cassandra.core.cql.Ordering;
import org.springframework.data.cassandra.core.cql.PrimaryKeyType;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKeyColumn;
import org.springframework.data.cassandra.core.mapping.Table;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@Table("tacos")                                          (1)
public class Taco {

  @PrimaryKeyColumn(type=PrimaryKeyType.PARTITIONED)     (2)
  private UUID id = Uuids.timeBased();

  @NotNull
  @Size(min = 5, message = "Name must be at least 5 characters long")
  private String name;

  @PrimaryKeyColumn(type=PrimaryKeyType.CLUSTERED,       (3)
          ordering=Ordering.DESCENDING)
  private Date createdAt = new Date();
```

```
    @Size(min=1, message="You must choose at least 1 ingredient")
    @Column("ingredients")                                    ❹
    private List<IngredientUDT> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
      this.ingredients.add(TacoUDRUtils.toIngredientUDT(ingredient));
    }
  }
```

❶ Persists to the "tacos" table

❷ Defines the partition key

❸ Defines the clustering key

❹ Maps the list to the "ingredients" column

As you can see, mapping the `Taco` class is a bit more involved. As with `Ingredient`, the `@Table` annotation is used to identify `tacos` as the name of the table that tacos should be written to. But that's the only thing similar to `Ingredient`.

The `id` property is still your primary key, but it's only one of two primary key columns. More specifically, the `id` property is annotated with `@PrimaryKeyColumn` with a `type` of `PrimaryKeyType.PARTITIONED`. This specifies that the `id` property serves as the partition key, used to determine to which Cassandra partition(s) each row of taco data will be written.

You'll also notice that the `id` property is now a `UUID` instead of a `Long`. Although it's not required, properties that hold a generated ID value are commonly of type `UUID`. Moreover, the `UUID` is initialized with a time-based UUID value for new `Taco` objects (but which may be overridden when reading an existing `Taco` from the database).

A little further down, you see the `createdAt` property that's mapped as another primary key column. But in this case, the `type` attribute of `@PrimaryKeyColumn` is set to `PrimaryKeyType.CLUSTERED`, which designates the `createdAt` property as a clustering key. As mentioned earlier, clustering keys are used to determine the ordering of rows *within a partition*. More specifically, the ordering is set to descending order—therefore, within a given partition, newer rows appear first in the `tacos` table.

Finally, the `ingredients` property is now a `List` of `IngredientUDT` objects instead of a `List` of `Ingredient` objects. As you'll recall, Cassandra tables are highly denormalized and may contain data that's duplicated from other tables. Although the `ingredient` table will serve as the table of record for all available `ingredients`, the ingredients chosen for a taco will be duplicated in the `ingredients` column. Rather than simply reference one or more rows in the `ingredients` table, the `ingredients` property will contain full data for each chosen ingredient.

But why do you need to introduce a new `IngredientUDT` class? Why can't you just reuse the `Ingredient` class? Put simply, columns that contain collections of data, such as the `ingredients` column, must be collections of native types (integers, strings, and so on) or user-defined types.

In Cassandra, user-defined types enable you to declare table columns that are richer than simple native types. Often they're used as a denormalized analog for relational foreign keys. In contrast to foreign keys, which only hold a reference to a row in another table, columns with user-defined types actually carry data that may be copied from a row in another table. In the case of the `ingredients` column in the `tacos` table, it will contain a collection of data structures that define the ingredients themselves.

You can't use the `Ingredient` class as a user-defined type, because the `@Table` annotation has already mapped it as an entity for persistence in Cassandra. Therefore, you must create a new class to define how ingredients will be stored in the `ingredients` column of the `taco` table. `IngredientUDT` (where `UDT` means *user-defined type*) is the class for the job, as shown here:

```
package tacos;

import org.springframework.data.cassandra.core.mapping.UserDefinedType;

import lombok.AccessLevel;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;

@Data
@RequiredArgsConstructor
@NoArgsConstructor(access = AccessLevel.PRIVATE, force = true)
@UserDefinedType("ingredient")
public class IngredientUDT {

  private final String name;
```

```
  private final Ingredient.Type type;

}
```

Although `IngredientUDT` looks a lot like `Ingredient`, its mapping requirements are much simpler. It's annotated with `@UserDefinedType` to identify it as a user-defined type in Cassandra. But otherwise, it's a simple class with a few properties.

You'll also note that the `IngredientUDT` class doesn't include an `id` property. Although it could include a copy of the `id` property from the source `Ingredient`, that's not necessary. In fact, the user-defined type may include any properties you wish—it doesn't need to be a one-to-one mapping with any table definition.

I realize that it might be difficult to visualize how data in a user-defined type relates to data that's persisted to a table. Figure 4.1 shows the data model for the entire Taco Cloud database, including user-defined types.

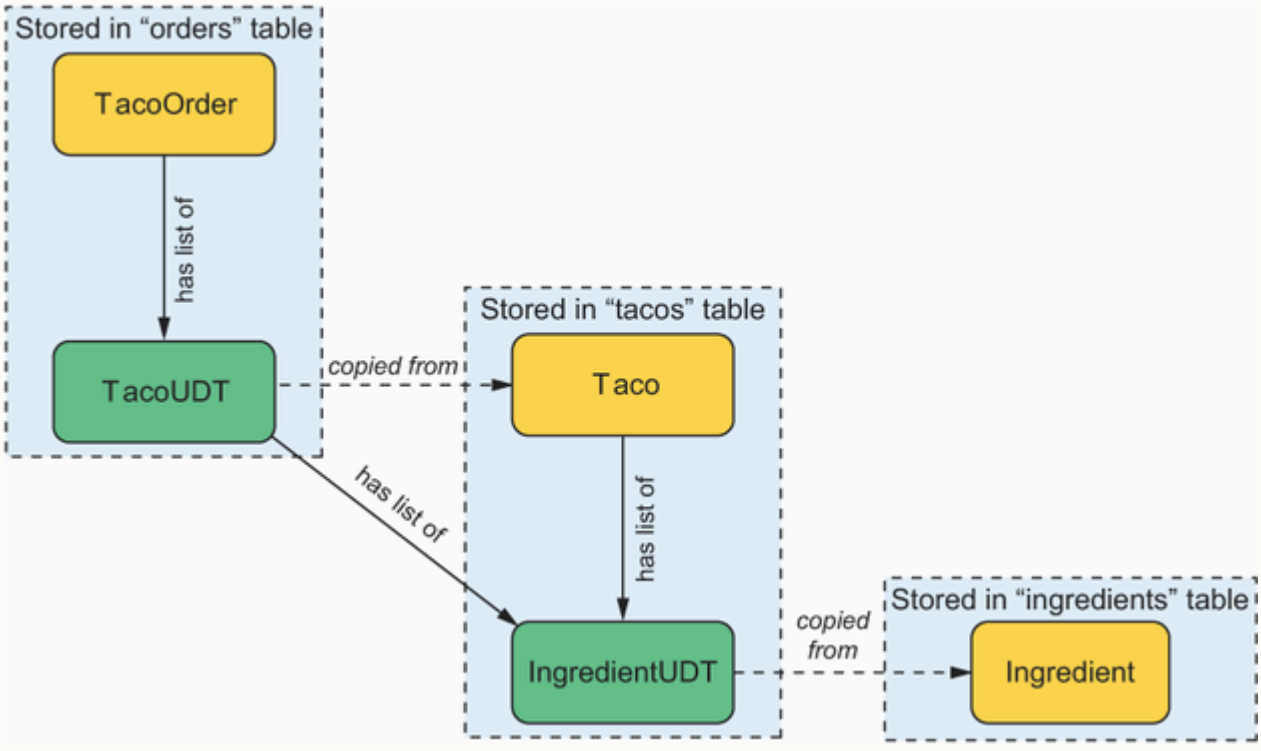


Figure 4.1 Instead of using foreign keys and joins, Cassandra tables are denormalized, with user-defined types containing data copied from related tables.

Specific to the user-defined type that you just created, notice how `Taco` has a list of `IngredientUDT` objects, which holds data copied from `Ingredient` objects. When a `Taco` is persisted, it's the `Taco` object and the list of `IngredientUDT` objects that's persisted to the `tacos` table. The list of `IngredientUDT` objects is persisted entirely within the `ingredients` column.

Another way of looking at this that might help you understand how user-defined types are used is to query the database for rows from the `tacos` table. Using CQL and the `cqlsh` tool that comes with Cassandra, you see the following results:

```
cqlsh:tacocloud> select id, name, createdAt, ingredients from tacos;

 id      | name      | createdat | ingredients
---------+-----------+-----------+-------------------------------------
 827390...| Carnivore | 2018-04...| [{name: 'Flour Tortilla', type: 'WRAP'},
                                  {name: 'Carnitas', type: 'PROTEIN'},
                                  {name: 'Sour Cream', type: 'SAUCE'},
                                  {name: 'Salsa', type: 'SAUCE'},
                                  {name: 'Cheddar', type: 'CHEESE'}]

(1 rows)
```

As you can see, the `id`, `name`, and `createdat` columns contain simple values. In that regard, they aren't much different than what you'd expect from a similar query against a relational database. But the `ingredients` column is a little different. Because it's defined as containing a collection of the user-defined `ingredient` type (defined by `IngredientUDT`), its value appears as a JSON array filled with JSON objects.

You likely noticed other user-defined types in figure 4.1. You'll certainly be creating some more as you continue mapping your domain to Cassandra tables, including some that will be used by the `TacoOrder` class. The next listing shows the `TacoOrder` class, modified for Cassandra persistence.

Listing 4.2 Mapping the `TacoOrder` class to a Cassandra `orders` table

```
package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;
import org.springframework.data.cassandra.core.mapping.Column;
```

```
import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@Table("orders")                                    (1)
public class TacoOrder implements Serializable {

  private static final long serialVersionUID = 1L;

  @PrimaryKey                                       (2)
  private UUID id = Uuids.timeBased();

  private Date placedAt = new Date();

  // delivery and credit card properties omitted for brevity's sake

  @Column("tacos")                                  (3)
  private List<TacoUDT> tacos = new ArrayList<>();

  public void addTaco(TacoUDT taco) {
    tacos.add(taco);
  }

}
```

(1) Maps to the orders table

(2) Declares the primary key

(3) Maps a list to the tacos column

Listing 4.2 purposefully omits many of the properties of `TacoOrder` that don't lend themselves to a discussion of Cassandra data modeling. What's left are a few properties and mappings, similar to how `Taco` was defined. `@Table` is used to map `TacoOrder` to the `orders` table, much as `@Table` has been used before. In this case, you're unconcerned with ordering, so the `id` property is simply annotated with `@PrimaryKey`, designating it as both a partition key and a clustering key with default ordering.

The `tacos` property is of some interest in that it's a `List<TacoUDT>` instead of a list of `Taco` objects. The relationship between `TacoOrder` and `Taco/TacoUDT` here is similar

to the relationship between `Taco` and `Ingredient/IngredientUDT`. That is, rather than joining data from several rows in a separate table through foreign keys, the `orders` table will contain all of the pertinent taco data, optimizing the table for quick reads.

The `TacoUDT` class is quite similar to the `IngredientUDT` class, although it does include a collection that references another user-defined type, as follows:

```
package tacos;

import java.util.List;
import org.springframework.data.cassandra.core.mapping.UserDefinedType;
import lombok.Data;

@Data
@UserDefinedType("taco")
public class TacoUDT {

  private final String name;
  private final List<IngredientUDT> ingredients;

}
```

Although it would have been nice to reuse the same domain classes you created in chapter 3, or at most to swap out some JPA annotations for Cassandra annotations, the nature of Cassandra persistence is such that it requires you to rethink how your data is modeled. But now that you've mapped your domain, you're ready to write repositories.

1.4 Writing Cassandra repositories

As you saw in chapter 3, writing a repository with Spring Data involves simply declaring an interface that extends one of Spring Data's base repository interfaces and optionally declaring additional query methods for custom queries. As it turns out, writing Cassandra repositories isn't much different.

In fact, there's very little that you'll need to change in the repositories we've already written to make them work for Cassandra persistence. For example, consider the following `IngredientRepository` we created in chapter 3:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;
```

```
import tacos.Ingredient;

public interface IngredientRepository
        extends CrudRepository<Ingredient, String> {

}
```

By extending `CrudRepository` as shown here, `IngredientRepository` is ready to persist `Ingredient` objects whose ID property (or, in the case of Cassandra, the primary key property) is a `String`. That's perfect! No changes are needed for `IngredientRepository`.

The changes required for `OrderRepository` are only slightly more involved. Instead of a `Long` parameter, the ID parameter type specified when extending `CrudRepository` will be changed to `UUID` as follows:

```
package tacos.data;

import java.util.UUID;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
        extends CrudRepository<TacoOrder, UUID> {

}
```

There's a lot of power in Cassandra, and when it's teamed up with Spring Data, you can wield that power in your Spring applications. But let's shift our attention to another database for which Spring Data repository support is available: MongoDB.

4.2 Writing MongoDB repositories

MongoDB is a another well-known NoSQL database. Whereas Cassandra is a column-store database, MongoDB is considered a document database. More specifically, MongoDB stores documents in BSON (Binary JSON) format, which can be queried for and retrieved in a way that's roughly similar to how you might query for data in any other database.

As with Cassandra, it's important to understand that MongoDB isn't a relational database. The way you manage your MongoDB server cluster, as well as how you model your data, requires a different mindset than when working with other kinds of databases.

That said, working with MongoDB and Spring Data isn't dramatically different from how you might use Spring Data for working with JPA or Cassandra. You'll annotate your domain classes with annotations that map the domain type to a document structure. And you'll write repository interfaces that very much follow the same programming model as those you've seen for JPA and Cassandra. Before you can do any of that, though, you must enable Spring Data MongoDB in your project.

12.2.1 Enabling Spring Data MongoDB

To get started with Spring Data MongoDB, you'll need to add the Spring Data MongoDB starter to the project build. As with Spring Data Cassandra, Spring Data MongoDB has two separate starters to choose from: one reactive and one nonreactive. We'll look at the reactive options for persistence in chapter 13. For now, add the following dependency to the build to work with the nonreactive MongoDB starter:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-mongodb
  </artifactId>
</dependency>
```

This dependency is also available from the Spring Initializr by checking the MongoDB check box under NoSQL.

By adding the starter to the build, autoconfiguration will be triggered to enable Spring Data support for writing automatic repository interfaces, such as those you wrote for JPA in chapter 3 or for Cassandra earlier in this chapter.

By default, Spring Data MongoDB assumes that you have a MongoDB server running locally and listening on port 27017. If you have Docker installed on your machine, an easy way to get a MongoDB server running is with the following command line:

```
$ docker run -p 27017:27017 -d mongo:latest
```

But for convenience in testing or developing, you can choose to work with an embedded Mongo database instead. To do that, add the following Flapdoodle embedded MongoDB dependency to your build:

```
<dependency>
  <groupId>de.flapdoodle.embed</groupId>
  <artifactId>de.flapdoodle.embed.mongo</artifactId>
  <!-- <scope>test</scope> -->
</dependency>
```

The Flapdoodle embedded database affords you all of the same convenience of working with an in-memory Mongo database as you'd get with H2 when working with relational data. That is, you won't need to have a separate database running, but all data will be wiped clean when you restart the application.

Embedded databases are fine for development and testing, but once you take your application to production, you'll want to be sure you set a few properties to let Spring Data MongoDB know where and how your production Mongo database can be accessed, as shown next:

```
spring:
  data:
    mongodb:
      host: mongodb.tacocloud.com
      port: 27017
      username: tacocloud
      password: s3cr3tp455w0rd
      database: tacoclouddb
```

Not all of these properties are required, but they're available to help point Spring Data MongoDB in the right direction in the event that your Mongo database isn't running locally. Breaking it down, here's what each property configures:

- `spring.data.mongodb.host` —The hostname where Mongo is running (default: `localhost`)
- `spring.data.mongodb.port` —The port that the Mongo server is listening on (default: `27017`)
- `spring.data.mongodb.username` —The username for accessing a secured Mongo database

- `spring.data.mongodb.password` —The password for accessing a secured Mongo database
- `spring.data.mongodb.database` —The database name (default: `test`)

Now that you have Spring Data MongoDB enabled in your project, you need to annotate your domain objects for persistence as documents in MongoDB.

2.2 Mapping domain types to documents

Spring Data MongoDB offers a handful of annotations that are useful for mapping domain types to document structures to be persisted in MongoDB. Although Spring Data MongoDB provides a half-dozen annotations for mapping, only the following four are useful for most common use cases:

- `@Id` —Designates a property as the document ID (from Spring Data Commons)
- `@Document` —Declares a domain type as a document to be persisted to MongoDB
- `@Field` —Specifies the field name (and, optionally, the order) for storing a property in the persisted document
- `@Transient` —Specifies that a property is not to be persisted

Of those three annotations, only the `@Id` and `@Document` annotations are strictly required. Unless you specify otherwise, properties that aren't annotated with `@Field` or `@Transient` will assume a field name equal to the property name.

Applying these annotations to the `Ingredient` class, you get the following:

```
package tacos;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Document
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
public class Ingredient {

  @Id
```

```
  private String id;
  private String name;
  private Type type;

  public enum Type {
    WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
  }

}
```

As you can see, you place the `@Document` annotation at the class level to indicate that `Ingredient` is a document entity that can be written to and read from a Mongo database. By default, the collection name (the Mongo analog to a relational database table) is based on the class name, with the first letter lowercase. Because you haven't specified otherwise, `Ingredient` objects will be persisted to a collection named `ingredient`. But you can change that by setting the `collection` attribute of `@Document` as follows:

```
@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Document(collection="ingredients")
public class Ingredient {
...
}
```

You'll also notice that the `id` property has been annotated with `@Id`. This designates the property as being the ID of the persisted document. You can use `@Id` on any property whose type is `Serializable`, including `String` and `Long`. In this case, you're already using the `String`-defined `id` property as a natural identifier, so there's no need to change it to any other type.

So far, so good. But you'll recall from earlier in this chapter that `Ingredient` was the easy domain type to map for Cassandra. The other domain types, such as `Taco`, were a bit more challenging. Let's look at how you can map the `Taco` class to see what surprises it might hold.

MongoDB's approach to document persistence lends itself very well to the domain-driven-design way of applying persistence at the aggregate root level. Documents in MongoDB tend to be defined as aggregate roots, with members of the aggregate as subdocuments.

What that means for Taco Cloud is that because `Taco` is only ever persisted as a member of the `TacoOrder`-rooted aggregate, the `Taco` class doesn't need to be annotated as a `@Document`, nor does it need an `@Id` property. The `Taco` class can remain clean of any persistence annotations, as shown here:

```
package tacos;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import lombok.Data;

@Data
public class Taco {

  @NotNull
  @Size(min=5, message="Name must be at least 5 characters long")
  private String name;

  private Date createdAt = new Date();

  @Size(min=1, message="You must choose at least 1 ingredient")
  private List<Ingredient> ingredients = new ArrayList<>();

  public void addIngredient(Ingredient ingredient) {
    this.ingredients.add(ingredient);
  }

}
```

The `TacoOrder` class, however, being the root of the aggregate, will need to be annotated with `@Document` and have an `@Id` property, as follows:

```
package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.Digits;
```

```
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.Data;

@Data
@Document
public class TacoOrder implements Serializable {

  private static final long serialVersionUID = 1L;

  @Id
  private String id;

  private Date placedAt = new Date();

  // other properties omitted for brevity's sake

  private List<Taco> tacos = new ArrayList<>();

  public void addTaco(Taco taco) {
    tacos.add(taco);
  }

}
```

For brevity's sake, I've snipped out the various delivery and credit card fields. But from what's left, it's clear that all you need is `@Document` and `@Id`, as with the other domain types.

Notice, however, that the `id` property has been changed to be a `String` (as opposed to a `Long` in the JPA version or a `UUID` in the Cassandra version). As I said earlier, `@Id` can be applied to any `Serializable` type. But if you choose to use a `String` property as the ID, you get the benefit of Mongo automatically assigning a value to it when it's saved (assuming that it's `null`). By choosing `String`, you get a database-managed ID assignment and needn't worry about setting that property manually.

Although there are some more-advanced and unusual use cases that require additional mapping, you'll find that for most cases, `@Document` and `@Id`, along with an occa-

sional `@Field` or `@Transient`, are sufficient for MongoDB mapping. They certainly do the job for the Taco Cloud domain types.

All that's left is to write the repository interfaces.

2.3 Writing MongoDB repository interfaces

Spring Data MongoDB offers automatic repository support similar to what's provided by the Spring Data JPA and Spring Data Cassandra.

You'll start by defining a repository for persisting `Ingredient` objects as documents. As before, you can write `IngredientRepository` to extend `CrudRepository`, as shown here:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
        extends CrudRepository<Ingredient, String> {

}
```

Wait a minute! That looks *identical* to the `IngredientRepository` interface you wrote in section 4.1 for Cassandra! Indeed, it's the same interface, with no changes. This highlights one of the benefits of extending `CrudRepository`—it's more portable across various database types and works equally well for MongoDB as for Cassandra.

Moving on to the `OrderRepository` interface, you can see in the following snippet that it's quite straightforward:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
        extends CrudRepository<TacoOrder, String> {

}
```

Just like `IngredientRepository`, `OrderRepository` extends `CrudRepository` to gain the optimizations afforded in its `insert()` methods. Otherwise, there's nothing terribly special about this repository, compared to some of the other repositories you've defined thus far. Note, however, that the ID parameter when extending `CrudRepository` is now `String` instead of `Long` (as for JPA) or `UUID` (as for Cassandra). This reflects the change we made in `TacoOrder` to support automatic assignment of IDs.

In the end, working with Spring Data MongoDB isn't drastically different from the other Spring Data projects we've worked with. The domain types are annotated differently. But aside from the ID parameter specified when extending `CrudRepository`, the repository interfaces are nearly identical.

Summary

- Spring Data supports repositories for a variety of NoSQL databases, including Cassandra, MongoDB, Neo4j, and Redis.
- The programming model for creating repositories differs very little across different underlying databases.
- Working with nonrelational databases demands an understanding of how to model data appropriately for how the database ultimately stores the data.

Previous chapter
< 3 Working with data

Next chapter
5 Securing Spring >